

Torus Meeting Intelligence

Documentação do projeto | Versão 1.0.0 | Revisão de 30/08/2026

01. Visão do produto

O Torus organiza transcrições de reuniões comerciais por vendedor. A partir de um arquivo JSON, identifica a intenção de cada fala, procura menções a produtos TOTVS e calcula indicadores que ajudam a escolher o próximo contato com o cliente.

O vendedor consulta apenas as reuniões atribuídas a ele. O gerente consulta todas as reuniões, vê a distribuição por vendedor e atribui novas análises a um responsável. A restrição é aplicada pelo servidor, não apenas pelos menus da tela.

O que está implementado

- Login com dois perfis e sessões com duração de oito horas.
- Envio de transcrição, histórico, busca e detalhes das análises.
- Classificação em cancelamento, objeção de preço, expansão, satisfação e fala neutra.
- Indicadores de risco, oportunidade, sentimento e participação por palavras.
- Identificação de Protheus, RM, Fluig, Datasul e Logix.
- Persistência local em SQLite e API HTTP.

Escopo e limites

A entrada precisa estar transcrita. O Torus não captura áudio, não participa de chamadas, não usa LLM e não envia o texto para serviços externos de análise. As recomendações vêm de regras fixas, não de geração de texto por IA.

Esta versão é uma demonstração funcional local. Usa contas com senhas públicas e um classificador avaliado em uma amostra pequena e sintética. Os resultados precisam de revisão humana; não há evidência de desempenho suficiente para decisões comerciais automáticas.

Como consultar este documento

Seção	Assunto
02 e 03	Instalação e uso por vendedores e gerentes
04 a 06	Arquitetura, formato de entrada e API
07 e 08	Modelo e cálculo dos indicadores
09 e 10	Banco de dados, autenticação e segurança
11	Operação, testes, problemas comuns e próximos passos

O conteúdo descreve os arquivos e comportamentos presentes nesta revisão. O esquema das requisições pode ser consultado em `/docs` com o servidor ligado.

02. Instalação e execução

Pré-requisitos

O backend declara Python 3.10 ou superior. O inicializador para Windows também exige o gerenciador de dependências uv no PATH. Na primeira execução, a preparação do ambiente pode precisar de internet. Depois de instaladas as dependências, a análise é local.

Abra um PowerShell na pasta do projeto e confira:

```
uv --version
cd backend
uv sync --extra dev
```

pyproject.toml declara as dependências e uv.lock registra as versões resolvidas. O ambiente fica em backend/.venv. Não é necessário Node.js para abrir a interface.

Uso normal no Windows

1. Na raiz do projeto, abra INICIAR_TORUS.cmd.
2. Aguarde o navegador abrir em <http://127.0.0.1:8000/>.
3. Mantenha o servidor ligado enquanto usa o painel.
4. Ao terminar, abra ENCERRAR_TORUS.cmd.

O inicializador executa a API em segundo plano e verifica /health. Fechar o navegador não para o servidor. Evite abrir index.html diretamente; use o endereço local servido pela API.

Desenvolvimento no terminal

Dentro de backend, execute:

```
uv run python -m uvicorn app.api:app --host 127.0.0.1 --port 8000 --reload
```

Nesse modo, mantenha o terminal aberto; Ctrl+C encerra o servidor. Não use esse comando e o inicializador ao mesmo tempo na porta 8000.

Contas de demonstração

Nome	Perfil	E-mail	Senha
Marina Costa	Gerente	manager@torus.ai	Torus@2026
Ana Souza	Vendedora	ana@torus.ai	Vendas@2026
Carlos Lima	Vendedor	carlos@torus.ai	Vendas@2026

Na inicialização, o banco recebe essas contas caso ainda não existam. Quando não há nenhuma reunião, são criadas três demonstrações: duas para Ana e uma para Carlos. Reiniciar não apaga o histórico nem redefine senhas de usuários já existentes.

03. Manual de uso

Entrar e navegar

Informe e-mail e senha e selecione **Abrir painel de reuniões**. Os atalhos de demonstração preenchem os campos; ainda é necessário enviar o formulário. A opção **Mostrar** permite conferir a senha digitada.

Área	Como usar
Visão geral	Consultar reuniões processadas, faixas de risco e prioridade sugerida
Minha carteira / Reuniões da equipe	Buscar cliente, título ou vendedor e abrir Ver análise
Nova análise	Enviar uma transcrição já preparada em JSON
Equipe comercial	Consultar indicadores por vendedor; disponível só para gerentes

Adicionar uma reunião

1. Abra **Nova análise**.
2. Preencha o título e o nome do cliente.
3. Se estiver no perfil gerente, selecione o vendedor responsável.
4. Selecione um arquivo .json no formato da seção 05, com até 1.000.000 bytes.
5. Selecione **Gerar análise da conta** e aguarde a confirmação.
6. Confira os indicadores e compare os sinais com a transcrição original.

O arquivo `backend/examples/sample_meeting.json` serve para experimentar o fluxo. Os campos preenchidos no formulário prevalecem sobre os correspondentes do JSON.

Permissões por perfil

Ação	Vendedor	Gerente
Consultar reuniões	Somente as próprias	Todas
Criar análise	Atribuída ao próprio usuário	Exige escolher vendedor ativo
Consultar lista de vendedores	Não	Sim
Abrir reunião de outro vendedor	Não	Sim

Ler os resultados e sair

Risco e oportunidade variam de 0 a 100. Apesar do símbolo de porcentagem na interface, são índices por regras e não chances comprovadas de cancelamento ou venda. A participação mede palavras, não duração de fala.

Os totais são calculados por reunião, não por cliente único. Risco alto não comprova uma renovação agendada. O histórico não possui edição ou exclusão pela interface; reenviar o arquivo cria outro registro.

Use **Sair da conta** para invalidar a sessão. Se ela expirar, entre novamente. Não há recuperação de senha ou cadastro de usuário nesta versão.

04. Arquitetura e organização

Caminho de uma análise

```
Navegador: index.html
-> API: valida sessão, perfil e transcrição
-> text_processing: intenção, sentimento e produtos por fala
-> meeting_analysis: indicadores e recomendação
-> storage: grava reunião e análises no SQLite
-> resposta HTTP: painel e detalhes da reunião
```

O frontend é HTML, CSS e JavaScript em um único arquivo, sem compilação. A API usa FastAPI e Pydantic; Uvicorn serve as rotas e a página inicial. O classificador usa a biblioteca padrão do Python e carrega um artefato JSON local.

Caminho	Responsabilidade
index.html	Login, navegação, formulários, tabelas e detalhes
backend/app/api.py	Rotas, validação, autenticação e dados de demonstração
backend/app/storage.py	SQLite, usuários, hashes, sessões e filtros de acesso
backend/app/intent_classifier.py	Normalização, características e inferência
backend/app/text_processing.py	Classificação e extração de sinais por fala
backend/app/meeting_analysis.py	Consolidação e recomendação da reunião
backend/models/intent_classifier.json	Pesos, classes, vieses e IDF do modelo implantado
backend/models/model_metadata.json	Identificação, hash, métricas resumidas e limitações
backend/examples e backend/tests	Exemplo de entrada e verificações automatizadas
backend/runtime	Banco, logs e identificador do servidor local
scripts	Inicialização e encerramento no Windows
docs	Documentação em Markdown e PDF

Dependências entre componentes

O processamento importa o classificador; a consolidação importa o processamento. A API coordena esses módulos e a persistência. O armazenamento não calcula indicadores. Consultar o histórico retorna a análise gravada, sem executar novamente o modelo.

O modelo fica em cache no processo após a primeira leitura. Ao substituí-lo por uma versão compatível, reinicie o servidor e mantenha os metadados correspondentes. Conjuntos de treino e relatórios completos não são necessários para executar o aplicativo.

05. Contrato da transcrição

A API recebe JSON codificado em UTF-8. conversation deve ter pelo menos uma fala e cada text precisa conter texto após a remoção de espaços nas extremidades.

```
{
  "meeting_id": 1042,
  "title": "Revisão do contrato",
  "customer_name": "Cliente de exemplo",
  "conversation": [
    {
      "speaker": "vendedor",
      "text": "Como está o uso do Protheus?"
    },
    {
      "speaker": "cliente",
      "text": "O valor ficou alto para nosso orçamento."
    }
  ]
}
```

Campo	Regra na API
meeting_id	Inteiro obrigatório; identifica a reunião na origem
title	Opcional, até 120 caracteres; a tela exige preenchimento
customer_name	Opcional, até 120 caracteres; a tela exige preenchimento
seller_id	Inteiro; exigido para gerente, resolvido pelo usuário para vendedor
conversation	Lista não vazia de objetos de fala
speaker	Texto de 1 a 80 caracteres; padrão unknown
text	Texto não vazio, preservado como conteúdo original

Use cliente e vendedor para identificar os participantes. Outros nomes são aceitos, mas somente falas com speaker igual a cliente, sem distinguir maiúsculas, entram nos sinais e no sentimento do cliente. Produtos e termos são coletados de todos os participantes.

Identificadores e envio de arquivo

meeting_id é o identificador recebido; record_id é a chave criada pelo Torus. A consulta de detalhes usa o segundo. O identificador de origem não é único: repetir o envio gera outra reunião.

No upload, a extensão deve ser .json e o limite é 1.000.000 bytes. Esse limite é específico de /analyze_meeting_file; as rotas de JSON direto não têm o mesmo limitador de tamanho implementado. Sem título ou cliente, o armazenamento usa Reunião {meeting_id} e Cliente não informado.

06. API HTTP

Endereço local: `http://127.0.0.1:8000`. A documentação interativa está em `/docs`; `/redoc` oferece outra visualização e `/openapi.json` retorna o contrato em JSON.

Rotas e acesso

Método e rota	Acesso	Resultado
GET /	Público	Interface web
GET /health	Público	Estado e tipo do modelo carregado
POST /auth/login	Público	Token e usuário para credenciais válidas
GET /auth/me	Autenticado	Identificação do usuário
POST /auth/logout	Bearer	Invalida o token enviado
GET /users/sellers	Gerente	Vendedores ativos
GET /meetings	Autenticado	Reuniões permitidas para o perfil
GET /meetings/{record_id}	Autenticado	Transcrição e análise armazenada
GET /model_metrics	Autenticado	Metadados e avaliação resumida
POST /analyze	Autenticado	Classificação e registro de fala isolada
POST /analyze_meeting	Autenticado	Análise e armazenamento de JSON
POST /analyze_meeting_file	Autenticado	Análise e armazenamento de arquivo multipart

Autenticação e resposta

Envie `{"email": "ana@torus.ai", "password": "Vendas@2026"}` em `/auth/login`. A resposta inclui `access_token`, `token_type` e `user`. Nas rotas protegidas, envie o cabeçalho `Authorization: Bearer <access_token>`. Não publique tokens.

A análise de reunião retorna `meeting_id`, `record_id`, `seller`, `summary` e `message_analysis`. O resumo contém indicadores, produtos, termos e ação recomendada. Cada fala contém texto original, intenção, sentimento e classificação com confiança e probabilidades.

A rota `/model_metrics` contém identificação, hash e `evaluation_summary`, sem comparação completa entre algoritmos. Não há rotas de cadastro de usuários, edição ou exclusão de reuniões.

Respostas de erro

Código	Situação principal
400	Arquivo inválido ou JSON fora do formato de upload
401	Login incorreto ou sessão ausente, inválida ou expirada
403 / 404	Área de gerente restrita / reunião inexistente ou fora do acesso
413	Arquivo ultrapassa o limite de upload
422	Campos inválidos ou responsável ausente/inválido
503	Modelo indisponível na verificação de saúde

07. Modelo de classificação

O aplicativo executa uma Regressão Logística multiclasse implementada em Python. Os pesos estão em `backend/models/intent_classifier.json`; não há treinamento durante a inicialização ou a análise de reuniões.

Classes produzidas

Rótulo da API	Significado
<code>churn_risk</code>	Sinal de cancelamento ou saída
<code>price_objection</code>	Objeção relacionada a preço ou orçamento
<code>upsell_opportunity</code>	Interesse em expansão, módulos ou licenças
<code>satisfaction</code>	Manifestação de satisfação
<code>neutral</code>	Fala classificada como neutra

Da fala à previsão

1. Normaliza Unicode, retira acentos, converte para minúsculas e mantém letras de a a z, números e espaços.
2. Extrai palavras, pares consecutivos e sequências de 3, 4 e 5 caracteres.
3. Divide a contagem de cada característica pela maior contagem da fala e multiplica pelo IDF armazenado.
4. Calcula a soma ponderada e o viés de cada classe; aplica softmax e escolhe o maior valor.

Características desconhecidas não entram no vetor. Uma entrada sem caracteres utilizáveis recebe `neutral`, com confiança 1, por regra especial; esse valor não indica que o modelo compreendeu a fala.

Resultado da avaliação registrada

Item	Valor
Origem e tamanho	100 frases sintéticas, 20 de cada classe
Separação	75 para treino e 25 para teste
Acurácia / recall macro	0,4800 / 0,4800
Precisão macro / F1 macro	0,5022 / 0,4720

As métricas resumidas e o hash SHA-256 dos pesos estão em `model_metadata.json`. O teste de integridade compara esse hash com o arquivo carregado. Os números descrevem o modelo entregue, não a qualidade de qualquer reunião enviada posteriormente.

Não há LLM, compreensão contextual de toda a conversa ou classificação multirrótulo. Uma fala recebe uma única classe, mesmo quando mistura temas. A amostra sintética é pequena e as confianças não foram calibradas em dados reais.

08. Indicadores e recomendações

O classificador identifica intenções; as regras em `meeting_analysis.py` transformam os resultados em índices comerciais. Os pesos abaixo são definidos no código, não aprendidos pelo modelo.

Cálculos

C é a soma das confianças de churn nas falas do cliente; U, a soma das confianças de expansão; O, a quantidade de objeções de preço do cliente; P, a quantidade de produtos diferentes citados por qualquer participante; S, o sentimento médio do cliente entre 0 e 1.

```
Risco = min(100, 45*C + 12*O + 80*max(0, 0.5-S))
Oportunidade = min(100, 45*U + 8*P)
Sentimento exibido = 100*S
```

Os três índices são arredondados a duas casas. Sem falas reconhecidas como `cliente`, o sentimento assume 0,5. Menções a produtos do vendedor também aumentam a oportunidade; uma menção isolada não comprova intenção de compra.

Sentimento, termos e participação

O sentimento usa listas de expressões positivas e negativas e busca por trechos do texto. Empates recebem neutro. A confiança da regra é limitada a 0,95; ela não vem da Regressão Logística. O valor de cada fala é $0.5 + \text{confiança}/2$ quando positivo e $0.5 - \text{confiança}/2$ quando negativo; neutro vale 0,5.

Essa busca não trata negações ou ambiguidades de forma confiável. Expressões sobrepostas podem afetar a contagem. A média usa apenas as falas do cliente.

A participação divide as palavras de cada papel pelo total de palavras de todos os participantes. Os dois valores podem somar menos de 1 se houver outros papéis. Os termos principais são as oito palavras mais frequentes após filtros simples; não são tópicos de um modelo semântico.

Prioridade das recomendações

As regras são verificadas de cima para baixo; a primeira atendida determina a ação.

Condição	Ação proposta
Risco ≥ 70	Plano de retenção e contato executivo em até 24 horas
Objeção de preço e risco ≥ 40	Rever proposta de valor e condições
Oportunidade ≥ 45	Preparar proposta de expansão
Risco ≥ 30	Entender insatisfação e combinar responsáveis
Demais casos	Registrar próximos passos e manter acompanhamento

Repetir falas ou aumentar a duração da transcrição pode elevar os índices; não há normalização pelo tamanho da reunião nem comprovação de conversão ou cancelamento. As ações propostas são sugestões para revisão humana.

09. Persistência e configuração

O banco padrão é `backend/runtime/torus.db`. A pasta `runtime` é local e fica fora do Git. Ela contém reuniões, usuários, sessões e registros operacionais; não faz parte das dependências instaláveis.

Estrutura do banco

Tabela	Conteúdo e relações
users	Nome, e-mail único, hash da senha, perfil, estado ativo e criação
sessions	Hash do token, usuário associado, criação e expiração
meetings	ID interno, ID de origem, vendedor, criador, título, cliente e JSONs da transcrição, resumo e análise
analyses	Texto, intenção, sentimento e sinal de churn; associação opcional a reunião e vendedor

Um vendedor pode ter várias reuniões; uma reunião tem várias análises de fala; uma sessão pertence a um usuário. As chaves estrangeiras são habilitadas por conexão. Os filtros de perfil são aplicados nas consultas de listagem e detalhe.

POST `/analyze` grava análises isoladas sem associação ao vendedor ou a uma reunião. Não existe uma rota para listá-las. A lista principal é ordenada pela data de criação e pelo ID, do mais recente para o mais antigo.

Alterar o local do banco

`TORUS_DB_PATH` pode apontar para outro arquivo. Defina a variável antes de iniciar o processo; o caminho é lido quando o módulo é importado. Exemplo, a partir de `backend`:

```
$env:TORUS_DB_PATH = 'C:\TorusDados\torus.db'
uv run python -m uvicorn app.api:app --host 127.0.0.1 --port 8000
```

Escolha um diretório gravável e protegido. Um caminho novo cria outro banco; não migra automaticamente o histórico existente. O código cria tabelas ausentes e acrescenta campos específicos de compatibilidade, mas não possui migrações versionadas completas.

Preservação dos dados

Para fazer uma cópia simples do SQLite, encerre o Torus antes de copiar o banco para um local seguro. Preserve também o código e o modelo correspondentes. Confirme o conteúdo da cópia antes de qualquer restauração; não substitua o banco ativo sem guardar a versão anterior.

Transcrições e resultados são armazenados como texto, sem criptografia implementada pela aplicação. A cópia de segurança exige o mesmo cuidado de acesso do banco original. Não envie `runtime` junto de uma distribuição pública do código.

10. Autenticação, acesso e cuidados

Proteções implementadas

- Senhas com PBKDF2-HMAC-SHA256, salt aleatório de 16 bytes e 310.000 iterações.
- Tokens gerados com `secrets.token_urlsafe(32)`; somente o hash SHA-256 fica no banco.
- Expiração de sessão em oito horas e invalidação pelo logout.
- Consulta de usuário ativo em cada requisição autenticada.
- Resposta genérica para e-mail ou senha incorretos.
- Vendedores limitados às reuniões do próprio ID, inclusive no acesso direto à API.

O navegador mantém o token em `sessionStorage` e o envia no cabeçalho Bearer. Não são usados cookies de sessão. Fechar a aba normalmente remove esse armazenamento local, mas apenas o logout invalida imediatamente o token no servidor.

Regras de autorização

Um vendedor que envia `seller_id` de outra pessoa continua tendo a análise atribuída a si. Para gerentes, o responsável deve existir, estar ativo e ter perfil vendedor. Pedir uma reunião de outro vendedor retorna 404; consultar a equipe como vendedor retorna 403.

Configuração local

O inicializador vincula o serviço a 127.0.0.1. O CORS permite `http://127.0.0.1:8000`, `http://localhost:8000` e a origem `null`, esta última para compatibilidade com a prévia local. CORS não substitui autenticação ou autorização.

As senhas de demonstração estão no código e na tela de acesso. Apagar apenas as contas do banco não desativa a demonstração: a inicialização recria contas ausentes. Uma implantação compartilhada exigiria alterar esse comportamento e definir uma gestão real de usuários.

O que ainda não está implementado

Não há recuperação ou troca de senha pela interface, autenticação multifator, limite de tentativas de login, renovação de sessão, trilha de auditoria completa, política de retenção de transcrições, criptografia de dados em repouso ou gestão de segredos para implantação.

Não exponha esta configuração de demonstração à internet. Uma implantação compartilhada precisa de revisão específica de acesso, transporte HTTPS, origens permitidas, armazenamento e tratamento dos dados de clientes. Esses itens são trabalho futuro, não garantias da versão atual.

11. Operação, verificação e manutenção

Logs e diagnóstico

O inicializador grava `server.out.log`, `server.err.log` e `server.pid` em `backend/runtime`. O encerrador usa o PID e verifica o processo do ambiente do Torus antes de pará-lo. Não encerre processos desconhecidos apenas porque usam a porta 8000.

Sintoma	Conferência inicial
Página não abre / falha de conexão	Iniciar <code>INICIAR_TORUS.cmd</code> , aguardar e usar o endereço HTTP local; conferir <code>server.err.log</code>
E-mail ou senha incorretos	Conferir os acessos e se as credenciais do banco foram alteradas
Sessão expirada	Entrar novamente; a sessão dura oito horas
Vendedor não vê reunião	Conferir o responsável atribuído; o isolamento é intencional
Gerente não consegue enviar	Selecionar um vendedor ativo
Erro 400, 413 ou 422	Conferir JSON, UTF-8, campos, falas e limite do arquivo
Porta já em uso	Encerrar o Torus pelo próprio script ou identificar o serviço concorrente
Modelo indisponível	Restaurar o artefato íntegro e os metadados; reiniciar

Verificações automatizadas

Dentro de backend, execute:

```
uv run --extra dev python -m unittest discover -s tests -v
uv run --extra dev ruff check .
```

Os testes de acesso usam um banco temporário, sem apagar o banco da aplicação. A suíte desta revisão tem 15 testes: login, isolamento de reuniões, atribuição por perfil, logout, painel, CORS local, entradas inválidas, saúde do modelo, metadados, integridade dos pesos e processamento de exemplos.

Esses testes verificam comportamentos específicos; não certificam segurança ou desempenho em produção. As dependências atuais podem emitir um aviso de descontinuação do cliente HTTP dos testes, mesmo quando a suíte passa.

Atualização e próximos passos

Antes de atualizar, preserve o banco. Instale dependências a partir de `uv.lock`, rode as verificações e reinicie. Se o modelo mudar, atualize metadados e hash; análises antigas permanecem armazenadas com os resultados originais.

As prioridades são avaliar transcrições reais adequadamente preparadas, calibrar indicadores, tratar falas ambíguas, criar gestão de usuários, ampliar limites e validações das entradas, acrescentar migrações e auditoria e definir retenção de dados. Não há integração com CRM, captura de áudio ou comparação temporal de contas implementada.